PRACTICAL – 03

AIM : To implement the A* search algorithm for solving the 8-puzzle problem using a heuristic function and observe its performance.

LAB THEORY :

The A* algorithm is an informed search algorithm that finds a path from an initial state to a goal state. It uses both the actual cost and an estimated cost to select the next state.
The evaluation function is:
$f(n) = g(n) + h(n)$
Where:

- $g(n)$ = cost from the initial state to the current state.
- $h(n)$ = estimated cost from the current state to the goal.
- $f(n)$ = total estimated cost.

For the 8-puzzle, common heuristics are:

1. Misplaced Tiles: Counts the number of tiles that are not in their correct position.
2. Manhattan Distance: Calculates the total distance of each tile from its correct position.

A better heuristic generally reduces the number of states explored.

CODE :

```
goal = "123456780"

def heuristic(state):
    count = 0
    for i in range(9):
        if state[i] != '0' and state[i] != goal[i]:
            count += 1
    return count
```

```
def astar(start):
    current = start

    while current != goal:
        print(current)

        zero = current.index('0')
        moves = []

        for x in [zero-1, zero+1, zero-3, zero+3]:
            if 0 <= x < 9:
                s = list(current)
                s[zero], s[x] = s[x], s[zero]
                s = ''.join(s)
                moves.append(s)

        current = min(moves, key=heuristic)

    print(current)
    print("Goal Reached!")

astar("123405678")
```

OUTPUT :

```
123405678
123450678
123456078
123456780
Goal Reached!
```

CONCLUSION : The A* algorithm successfully solves the 8-puzzle using g(n) + h(n). The heuristic helps the algorithm reach the goal with fewer unnecessary searches.